Assignment No. : 04

Q1. Docker Compose: Nginx Static Website

Create a Docker Compose setup to run an Nginx container that serves a custom HTML page.

Requirements

1. Create a folder named website
2. Create a file index.html with content:

```
Hello from Docker Compose
```

```
student@student-VMware-Virtual-Platform:~$ cd website
student@student-VMware-Virtual-Platform:~/website$ cat > index.html
<h1> Hello from Docker Compose </h1>
student@student-VMware-Virtual-Platform:~/website$ nano docker-compose.yml
```

3. Use Docker Compose to:

   Run an Nginx container

   Map port 8080 to 80

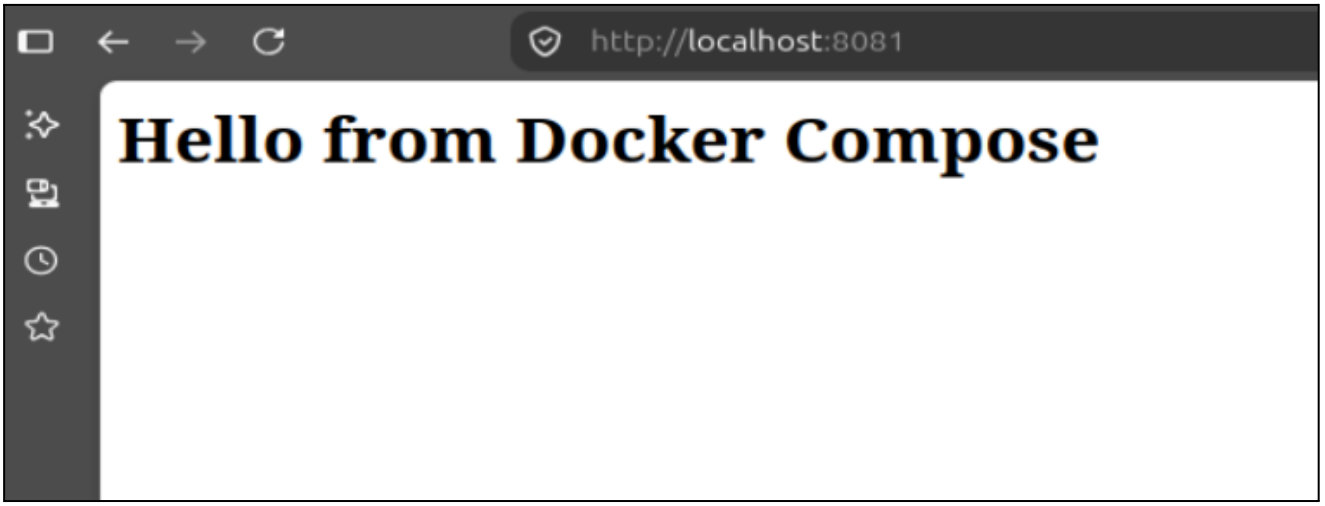
   Mount website folder to /usr/share/nginx/html

```
  GNU nano 8.7.1
version: '3.8'

services:
  web:
    image: nginx:latest
    ports:
      - "8081:80"
    volumes:
      - .:/usr/share/nginx/html
```

```
student@student-VMware-Virtual-Platform:~/website$ docker compose up
WARN[0000] /home/student/website/docker-compose.yml: the attribute `version` is obsc
ove it to avoid potential confusion
[+] up 1/1
 ✔ Container website-web-1 Recreated
Attaching to web-1
web-1  | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to
web-1  | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
web-1  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-
web-1  | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/c
web-1  | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx
web-1  | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.env
web-1  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templa
web-1  | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-proces
web-1  | /docker-entrypoint.sh: Configuration complete; ready for start up
```

4. Open browser and verify output



Q2. Docker Compose: Python Script Container

Question

Run a simple Python script using Docker Compose.

Requirements

1. Create a file app.py: print("Hello from Python Container")

```
student@student-VMware-Virtual-Platform:~$ mkdir py_docker
student@student-VMware-Virtual-Platform:~$ cd py_docker/
student@student-VMware-Virtual-Platform:~/py_docker$ cat > app.py
print("Hello from Python Container")
```

2. Create a Dockerfile to run this script

```
  GNU nano 8.7.1
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
CMD ["python","app.py"]
```

3. Use Docker Compose to run this container

```
  GNU nano 8.7.1
version: '3.8'
services:
  app:
    build: .
    container_name: my_py_app
```

4. Container should execute script and exit

```
 => resolving provenance for metadata file
[+] up 3/3
 ✔ Image py_docker-app        Built
 ✔ Network py_docker_default  Created
 ✔ Container my_py_app        Created
Attaching to my_py_app
my_py_app  | Hello from Python Container
my_py_app exited with code 0
```

Q3. Docker Compose: MySQL with Volume

Question

Create a MySQL container using Docker Compose with persistent storage.

Requirements

1. Use MySQL image
2. Set:

   root password

   database name

3. Create a named volume

```
 GNU nano 8.7.1                                  docker-compos
version: '3.8'
services:
  db:
    image: mysql:latest
    container_name: mysql_docker
    environment:
      MYSQL_ROOT_PASSWORD: "root123"
      MYSQL_DATABASE: cdac_db
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

4. Start container and:

   Enter MySQL using docker exec

   Create table test

```
student@student-VMware-Virtual-Platform:~/my_sql$ docker compose up -d
WARN[0000] /home/student/my_sql/docker-compose.yml: the attribute `version` is obsolete, it will
ve it to avoid potential confusion
[+] up 3/3
 ✓ Network my_sql_default Created
 ✓ Volume my_sql_db_data  Created
 ✓ Container mysql_docker Started
```

```
student@student-VMware-Virtual-Platform:~/my_sql$ docker exec -it mysql_docker mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 26.10.0 MySQL Community Server - GPL
```

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| cdac_db            |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.006 sec)
```

```
mysql> use cdac_db;
Database changed
mysql> create table data (name varchar(20),roll int);
Query OK, 0 rows affected (0.013 sec)

mysql> insert into data values ("babu",123);
Query OK, 1 row affected (0.011 sec)

mysql> select * from data;
+------+------+
| name | roll |
+------+------+
| babu |  123 |
+------+------+
```

5. Restart container and verify data still exists

```
student@student-VMware-Virtual-Platform:~/my_sql$ docker compose down
WARN[0000] /home/student/my_sql/docker-compose.yml: the attribute `version` is obsolete, it will be
ve it to avoid potential confusion
[+] down 2/2
 ✓ Container mysql_docker Removed
 ✓ Network my_sql_default Removed
```

```
student@student-VMware-Virtual-Platform:~/my_sql$ docker compose up -d
WARN[0000] /home/student/my_sql/docker-compose.yml: the attribute `version` is obsolete, it will be
ve it to avoid potential confusion
[+] up 2/2
 ✓ Network my_sql_default Created
 ✓ Container mysql_docker Started
student@student-VMware-Virtual-Platform:~/my_sql$ docker exec -it mysql_docker mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 26.10.0 MySQL Community Server - GPL
```

```
mysql> use cdac_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from data;
+------+------+
| name | roll |
+------+------+
| babu |  123 |
+------+------+
1 row in set (0.000 sec)
```

Q4. Docker Compose: One Volume with 3 Nginx Containers

Create a Docker Compose setup using three Nginx containers that share a single volume but display different HTML content.

Requirements

1. Create one named volume:

   shared-data

2. Create three Nginx containers:

   nginx1
   nginx2
   nginx3

3. Attach the same volume to all containers at:

   /usr/share/nginx/html

4. Each container should display different HTML content:

   nginx1 → Hello from Container 1
   nginx2 → Hello from Container 2
   nginx3 → Hello from Container 3

5. Expose each container on different ports:

   nginx1 → 8081
   nginx2 → 8082
   nginx3 → 8083

```
student@student-VMware-Virtual-Platform:~/nginx_vol$ ls webpages/
index1.html  index2.html  index3.html
```

```
 GNU nano 8.7.1                                  docker-compose.yml
version: '3.8'
services:
  web1:
    image: nginx:latest
    container_name: nginx1
    ports:
      - "8081:80"
    volumes:
      - shared_data:/usr/share/nginx/html
      - ./webpages/index1.html:/usr/share/nginx/html/index.html

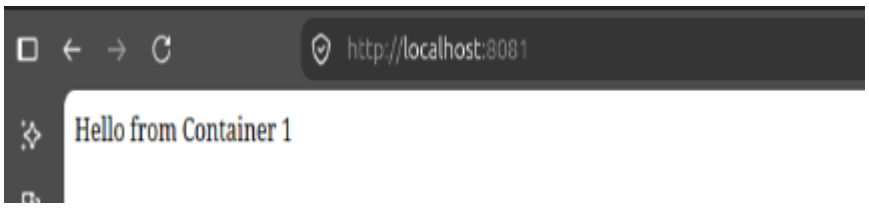
  web2:
    image: nginx:latest
    container_name: nginx2
    ports:
      - "8082:80"
    volumes:
      - shared_data:/usr/share/nginx/html
      - ./webpages/index2.html:/usr/share/nginx/html/index.html
```

```
  web3:
    image: nginx:latest
    container_name: nginx3
    ports:
      - "8083:80"
    volumes:
      - shared_data:/usr/share/nginx/html
      - ./webpages/index3.html:/usr/share/nginx/html/index.html

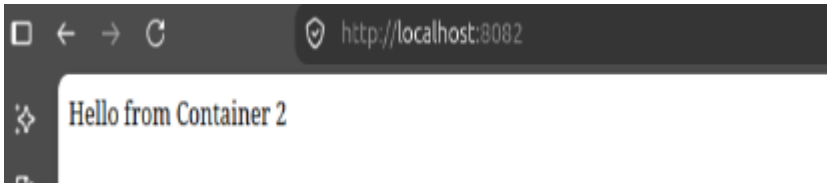
volumes:
  shared_data:
```

6. Ensure all three containers run simultaneously using Docker Compose. 7. Open browser and verify:

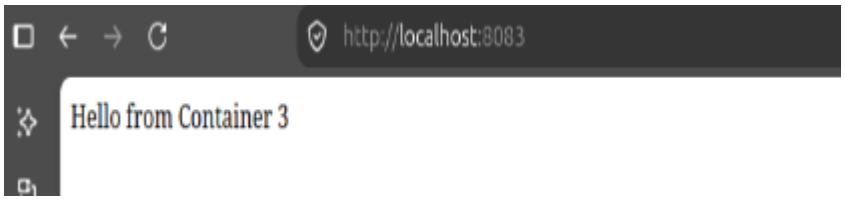
http://localhost:8081



http://localhost:8082



http://localhost:8083



Each port should show different content.

Q. 5 Write a steps for kubernetes Installation (Minikube)

Installing minikube (Ubuntu)

Task	Command
Download minikube	`curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64`
Install it	`sudo install minikube-linux-amd64 /usr/local/bin/minikube`
Let your user run Docker (once)	`sudo usermod -aG docker $USER && newgrp docker`
Start the cluster (first run takes a few minutes)	`minikube start --driver=docker`
Check the cluster is running	`minikube status`
Check the node is Ready	`minikube kubectl -- get nodes`
Create a kubectl shortcut	`echo 'alias kubectl="minikube kubectl --"' >> ~/.bashrc`
Load the shortcut	`source ~/.bashrc`
Verify	`kubectl get nodes`